Sheet 2

Registers · Data Definition · DUP · Questions

I. **Part A — Written Questions.** 25 questions on identifiers, `BYTE`/`WORD`/`DWORD` declarations, `DUP`, `$` and little-endian order.

II. **Part B — Multiple Choice.** 40 questions on registers, flags, CPU modes and assembly syntax.

III. **Part C — True or False.** 45 statements.

Notation. [L1, S28] means Lecture 1, slide 28. [L2, S25] means Lecture 2, slide 25.
[Section note] marks material that the sheet uses but Lectures 1–2 do not cover (data definition, `DUP`, `$`, `EQU`, `MOVZX`/`MOVSX` …).

Mahmoud AboZied
Teaching Assistant

Academic Year 2026 / 2027

Contents

Part A · Written Questions **3**

Part B · Multiple Choice **7**

Part C · True or False **10**

Part A · Written Questions

Part A · Written Questions (25 questions) Name: ..... ID: .....

Question 1
Identify the valid and invalid identifiers:

`NUMBER1` `2NUMBER` `MY_VAR` `MY-VAR` `COUNT` `ADD`

Questions 2 – 6
What does the **highlighted value** mean in each of the following declarations?[1]

Q	Declaration	Meaning
2	`value1 BYTE ?`	— the `?`
3	`value2 BYTE 0`	— the `0`
4	`value3 BYTE 255`	— the `255`
5	`value4 SBYTE -128`	— the `-128`
6	`value5 SBYTE +127`	— the `+127`

Question 7
Which of the following is an **invalid** identifier?

(a) COUNT (b) SUM1 (c) 1SUM (d) TOTAL_SUM

Question 8
Consider the following declaration:

```
str3 BYTE 'A', 'E', 'I', 'O', 'U'
```

a. How many bytes does `str3` occupy?
b. What are the values stored in `str3`?
c. What is the first character in `str3`?
d. What is the last character in `str3`?
e. What does `BYTE` mean in this declaration?
f. How many characters are stored in `str3`?
g. What type of characters are stored in `str3`?

Question 9
Consider the following declaration:

[1] The original sheet does not print the declarations for Questions 2–6, 17 and 20. The ones used here are the standard textbook examples that the questions refer to.

```
var5 BYTE 3, 2 DUP(1,2), 3, 3 DUP(2 DUP(5,6))
```

a. How many bytes does `var5` occupy?
b. What are the values stored in `var5`?
c. What does `2 DUP(1,2)` mean?
d. What does `3 DUP(2 DUP(5,6))` mean?[2]

Question 10
Consider the following declaration:

```
var5 BYTE ?, 2 DUP(1,2), 3, 3 DUP(2 DUP(0))
```

a. How many bytes does `var5` occupy?
b. What are the values stored in `var5`?
c. What does `2 DUP(1,2)` mean?
d. What does `3 DUP(2 DUP(0))` mean?

Question 11
What does `DUP` mean? `array BYTE 5 DUP(0)`

Question 12
Calculate the size: `var1 BYTE 5, 2 DUP(10), 3, 4 DUP(0)`

Question 13
Calculate the size: `list WORD 1000h, 2000h, 3000h, 4000h`

Question 14
What is the value of `ListSize`?

```
list     WORD 1000h, 2000h, 3000h, 4000h
ListSize = ($ - list) / 2
```

Question 15
What does `$` mean in assembly language?

[2] The original sheet asks about `3DUP (2 DUP(5))`, but the declaration contains `3 DUP(2 DUP(5,6))`. The key answers both.

Question 16
What is the size of this string? `myString BYTE "HELLO"`

Question 17
What is the size? `myString BYTE "HELLO", 0`

Question 18
Which is a **valid** identifier?

(a) 2VALUE (b) MY-VALUE (c) VALUE2 (d) MY VALUE

Question 19
What is stored here? `value1 BYTE 'A'`

Question 20
What does `?` mean? `value1 BYTE ?`

Question 21
What does this mean? `value1 BYTE 255`

Question 22
What does this mean? `value1 SBYTE -128`

Question 23
Calculate the values and the size: `var2 BYTE 7, 3 DUP(5), 2, 2 DUP(0)`

Question 24
Show how this value is stored in **little-endian** order: `value DWORD 12345678h`

Address	Byte
Lowest (`value + 0`)	
`value + 1`	
`value + 2`	
Highest (`value + 3`)	

Question 25
Explain the difference between the following declarations:

```
COUNT = 10
NUM   BYTE 10
```

Part B · Multiple Choice

Part B · Multiple Choice (40 questions) Name: ..... ID: .....

Choose the one best answer for each question.[3]

1. What is the purpose of the BX register in the Intel 8086 microprocessor?
(a) Arithmetic operations (b) Base register for memory addressing
(c) Loop counting (d) Stack management

2. Which register is often used as a loop counter in the 8086?
(a) AX (b) BX (c) CX (d) DX

3. The AH and AL registers in the 8086 represent:
(a) High-order and low-order halves of AX (b) Accumulator and Base registers
(c) Address and Data registers (d) Index and Pointer registers

4. What is the primary function of the CX register in the 8086?
(a) Arithmetic operations (b) Base register for memory addressing
(c) Loop counting (d) Stack management

5. Which segment register is associated with the stack in the 8086?
(a) CS (b) DS (c) SS (d) ES

6. The SP register in the 8086 is used for:
(a) Arithmetic operations (b) Stack management
(c) Loop counting (d) Memory addressing

7. What is the role of the IP register in the 8086?
(a) Arithmetic operations
(b) Instruction pointer for program execution
(c) Stack management
(d) Data transfer

8. The 8086 has how many general-purpose registers?
(a) 2 (b) 4 (c) 6 (d) 8

9. Which of the following is a special-purpose register in the 8086?
(a) AX (b) CX (c) SP (d) BX

10. Memory in the 8086 is divided into segments. Which register holds the address of the current code segment?
(a) CS (b) DS (c) SS (d) ES

11. The 8086's 16-bit architecture means it can process:
(a) 8 bits of data at a time (b) 16 bits of data at a time
(c) 32 bits of data at a time (d) 64 bits of data at a time

12. What is the purpose of the BX register in the 8086?
(a) Arithmetic operations (b) Base register for memory addressing
(c) Loop counting (d) Stack management

13. Which segment register is associated with the data segment in the 8086?
(a) CS (b) DS (c) SS (d) ES

14. The 8086 register used as a base pointer for memory addressing is:
(a) SP (b) IP (c) BP (d) SI

15. In the 8086, the high- and low-order halves of the AX register are represented by:
(a) AH and BL (b) AL and BH (c) AH and AL (d) BH and BL

16. Which of the following is a general-purpose register in the 8086?
(a) CS (b) IP (c) SP (d) BX

[3] The original sheet skips number 22. The questions here are renumbered 1–40, so original Q23–Q41 are Q22–Q40.

17. The 8086 instruction pointer (IP) is automatically incremented after each instruction execution.
(a) True (b) False

18. What is the primary function of the CX register in the 8086?
(a) Arithmetic operations (b) Loop counting
(c) Base register for memory addressing (d) Stack management

19. Which register is often used as a base register for memory addressing in the 8086?
(a) CX (b) SP (c) SI (d) BX

20. The 8086 segment register that holds the address of the stack segment is:
(a) CS (b) DS (c) SS (d) ES

21. The clock cycle:
(a) Synchronizes CPU and bus operation
(b) Indicates the time of a single operation
(c) Triggers the events
(d) All above

22. AH, AL and AX registers are ....., ... and ..... bits.
(a) 8, 16, 8 (b) 8, 8, 16 (c) 16, 16, 8 (d) 16, 8, 8

23. CX, EBX and DH are ....., ... and ..... bits.
(a) 16, 32, 8 (b) 16, 16, 8 (c) 16, 32, 16 (d) 16, 8, 32

24. The accumulator register EAX is:
(a) Instruction register (b) Segment register
(c) General-purpose register (d) 16-bit register

25. The parity flag checks that the:
(a) Unsigned arithmetic is out of range (b) Result is negative
(c) Sum of 1 bits is an even number (d) Carry is from bit 3 to bit 4

26. Floating-point registers are:
(a) Eight 16-bit floating-point data registers
(b) Eight 32-bit floating-point data registers
(c) Eight 64-bit floating-point data registers
(d) Eight 80-bit floating-point data registers

27. A complex instruction set (CISC) has:
(a) Simple instructions (b) High-level operations
(c) Small instructions (d) All above

28. A reduced instruction set (RISC) is:
(a) A large instruction set (b) Requires a microcode interpreter
(c) Directly executed by hardware (d) All above

29. Real-address mode is:
(a) Supported by the MS-DOS operating system
(b) Designed for multitasking
(c) Supported by Linux & MS-Windows
(d) All above

30. Protected mode is:
(a) Supported by the MS-DOS operating system
(b) Designed for multitasking
(c) Application programs can access any area of memory
(d) All above

31. When a HLL program displays a string of characters, these steps take place: 1 – Application program, 2 – BIOS function, 3 – OS function, 4 – Hardware. The order is:
(a) 1, 2, 3, 4 (b) 1, 2, 4, 3 (c) 1, 3, 4, 2 (d) 1, 3, 2, 4

32. The minus operator (–) has ..... precedence than the division operator (/) in integer expressions.
(a) The same (b) Higher (c) Lower (d) Any

33. The enclosed string 'ABC' occupies:
(a) 3 bytes (b) 2 bytes (c) 1 byte (d) 4 bytes

34. 'Say "Goodbye," Diana' is:
(a) An embedded-quotes string expression (b) A valid string expression
(c) Not a valid string expression (d) Any of the above

35. Reserved words in assembly:
(a) Can be used as identifiers
(b) Are operators
(c) First character must be a letter
(d) Any of the above

36. Identifiers:
(a) Can be used as reserved words
(b) Are operators
(c) First character must be a letter
(d) Any of the above

37. Identifiers have:
(a) 1–249 characters including digits
(b) 1–249 characters not including digits
(c) 1–247 characters not including digits
(d) 1–247 characters including digits

38. An instruction contains a label (……) and a mnemonic (……).
(a) Required, required
(b) Optional, required
(c) Optional, optional
(d) Required, optional

39. MOV, ADD and SUB are:
(a) Constant expressions
(b) Labels
(c) Reserved words
(d) Identifiers

40. Comments:
(a) Are required
(b) Begin with a semicolon (;)
(c) Start the program
(d) End the program

Part C · True or False

Part C · True or False (45 statements) Name: ..... ID: .....

*Circle **T** if the statement is true and **F** if it is false.*

1. `list WORD 1000h,2000h,3000h,4000h` occupies 8 bytes. (T) (F)

2. `list DWORD 1,2,3,4` contains 4 elements. (T) (F)

3. Registers are slower than conventional RAM. (T) (F)

4. The zero flag (ZF) is set when an arithmetic operation produces a zero result. (T) (F)

5. The identifier `123Var` is valid in assembly language. (T) (F)

6. Assembly language is case-sensitive when defining identifiers. (T) (F)

7. A label in assembly language can be used to mark the location of variables or instructions. (T) (F)

8. Each character in a string constant occupies a single byte in memory. (T) (F)

9. The `ADD` instruction is used for subtracting one value from another. (T) (F)

10. The `CALL` instruction is used for calling a procedure. (T) (F)

11. The result of the `CLC` instruction is CF = 0. (T) (F)

12. `DUP` is used to repeat a value or group of values. (T) (F)

13. `3 DUP(5)` means 3, 3, 3, 3, 3. (T) (F)

14. `3 DUP(5)` means 5, 5, 5. (T) (F)

15. `var1 BYTE 5, 2 DUP(10), 3, 4 DUP(0)` occupies 8 bytes. (T) (F)

16. `$ - list` can be used to calculate the size of an array in bytes. (T) (F)

17. For a `WORD` array, dividing `$ - list` by 2 gives the number of elements. (T) (F)

18. For a `DWORD` array, dividing `$ - list` by 2 gives the number of elements. (T) (F)

19. `list DWORD 10000000h,20000000h` has a total size of 8 bytes. (T) (F)

20. `MOV` copies data from the source operand to the destination operand. (T) (F)

21. `mov al, myBytes` is valid if `myBytes` is a `BYTE`. (T) (F)

22. `mov ax, myBytes` is valid if `myBytes` is a `BYTE`. (T) (F)

23. `MOVZX` converts a `BYTE` to a larger unsigned value. (T) (F)

24. `MOVZX` stands for Move with Zero Extension. (T) (F)

25. `MOVSX` stands for Move with Sign Extension. (T) (F)

26. `MOVSX` and `MOVZX` always produce the same result. (T) (F)

27. If `arrayW` is a `WORD` array, then `[arrayW+4]` points to the third element. (T) (F)

28. If `arrayD` is a `DWORD` array, `[arrayD+4]` points to the second element. (T) (F)

29. `MUL` performs unsigned multiplication. (T) (F)

30. `3 DUP(1,2)` creates 1, 2, 1, 2, 1, 2. (T) (F)

31. `2 DUP(1,2,3)` creates six values. (T) (F)

32. `arr BYTE 2 DUP(1)` followed by `WORD 2000h` occupies 6 bytes. (T) (F)

33. `arr BYTE 2 DUP(1)` followed by `WORD 2000h` and `BYTE 3` occupies 6 bytes. (T) (F)

34. `.data` is used for defining variables and initialized data. (T) (F)

35. A constant declared with `EQU` normally occupies memory. (T) (F)

36. `EQU` defines a symbolic constant; it doesn't allocate storage for the value. (T) (F)

37. `MAX EQU 100` defines a symbolic constant. (T) (F)

38. `arr BYTE 10, 2 DUP(20), 30` occupies 4 bytes. (T) (F)

39. `arr WORD 10, 2 DUP(20), 30` occupies 4 bytes. (T) (F)

40. For `arr DWORD 1,2,3`, the number of elements = `($ - arr) / 4`. (T) (F)

41. If an array contains 5 `DWORD` values, its total size is 20 bytes. (T) (F)

42. If an array occupies 20 bytes and consists of `WORD` values, it contains 20 elements. (T) (F)

43. If `arr` is a `BYTE` array, `[arr+3]` refers to the fourth element. (T) (F)

44. If `arr` is a `WORD` array, `[arr+3]` refers to the fourth element. (T) (F)

45. If `arr` is a `DWORD` array, `[arr+8]` refers to the third element. (T) (F)

— End of Sheet 2 —